

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO PROJECT

Hệ thống Phân tích và Xử lý dữ liệu Binance

HỌC PHẦN: LƯU TRỮ VÀ XỬ LÝ DỮ LIỆU LỚN - IT4931

Giảng viên hướng dẫn: TS. Trần Việt Trung

Nhóm thực hiện: Nhóm 5

Thành viên: Lưu Quốc Dũng – 20235302

Lê Hoàng Phúc – 20235190

Nguyễn Duy Liêm – 20239750

Mai Tiến Hoàng – 20235085

Nguyễn Khắc Nhất – 20235391

Mã lớp: 168482

Hà Nội, tháng 6 năm 2025

Mục lục

1	Định nghĩa bài toán	4
1.1	Bài toán lựa chọn	4
1.2	Phân tích mức độ phù hợp với dữ liệu lớn	5
1.3	Phạm vi và giới hạn dự án	6
2	Kiến trúc và thiết kế	7
2.1	Kiến trúc tổng thể	7
2.2	Data Ingestion Layer	8
2.2.1	Binance WebSocket API	9
2.2.2	Binance REST API	9
2.2.3	Kafka Message Broker	10
2.3	Processing & Lakehouse Layer	10
2.3.1	Luồng xử lý Medallion Architecture	11
2.3.2	Thiết kế Schema dữ liệu	13
2.3.3	Cơ chế xử lý Streaming	13
2.4	Serving & Alert Layer	14
2.4.1	FastAPI Alert Engine	14
2.4.2	Alert Evaluation Flow	14
2.4.3	Dashboard Visualization	15
2.5	Monitoring Layer	15
2.6	APScheduler Orchestration	16
2.7	Deploy Architecture	16
3	Chi tiết triển khai	17
3.1	Phân hệ thu thập dữ liệu (Data Ingestion)	17
3.2	Triển khai Kafka Message Queue	18
3.3	Triển khai xử lý dữ liệu với Apache Spark	19
3.3.1	Bronze Processing	19
3.3.2	Silver Processing	20
3.3.3	Gold Processing	20
3.4	Triển khai lưu trữ Lakehouse	21
3.5	Triển khai Alert Engine và Serving API	22

3.6	Triển khai Workflow với APScheduler	23
3.7	Triển khai hạ tầng Docker/Kubernetes	23
3.8	Monitoring và kiểm thử hệ thống	24
4	Bài học kinh nghiệm	24
4.1	Bài học 1: Đảm bảo tính liên tục khi thu thập dữ liệu Binance	24
4.1.1	Mô tả vấn đề	24
4.1.2	Cách tiếp cận đã thử	25
4.1.3	Giải pháp cuối cùng	25
4.1.4	Điểm rút ra	26
4.2	Bài học 2: Đồng bộ trạng thái giữa Kafka và Spark Structured Streaming .	26
4.2.1	Mô tả vấn đề	26
4.2.2	Cách tiếp cận đã thử	26
4.2.3	Giải pháp cuối cùng	26
4.2.4	Kết quả	27
4.2.5	Điểm rút ra	27
4.3	Bài học 3: Thiết kế Lakehouse thay vì lưu toàn bộ dữ liệu vào Database . .	27
4.3.1	Mô tả vấn đề	27
4.3.2	Cách tiếp cận đã thử	28
4.3.3	Giải pháp cuối cùng	28
4.3.4	Kết quả	28
4.3.5	Điểm rút ra	28
4.4	Bài học 4: Quản lý Small Files và tối ưu Delta Lake	29
4.4.1	Mô tả vấn đề	29
4.4.2	Cách tiếp cận đã thử	29
4.4.3	Giải pháp cuối cùng	29
4.4.4	Điểm rút ra	29
4.5	Bài học 5: Tối ưu tính toán Technical Indicator trên Spark	30
4.5.1	Mô tả vấn đề	30
4.5.2	Giải pháp cuối cùng	30
4.5.3	Kết quả	30
4.5.4	Điểm rút ra	30
4.6	Bài học 6: Thiết kế dữ liệu phục vụ Dashboard và Alert Engine	30

4.6.1	Mô tả vấn đề	30
4.6.2	Giải pháp cuối cùng	31
4.6.3	Kết quả	31
4.6.4	Điểm rút ra	31
4.7	Bài học 7: Quản lý cấu hình và triển khai Container	31
4.7.1	Mô tả vấn đề	31
4.7.2	Giải pháp cuối cùng	31
4.7.3	Kết quả	32
4.7.4	Điểm rút ra	32
4.8	Bài học 8: Monitoring Streaming Pipeline	32
4.8.1	Mô tả vấn đề	32
4.8.2	Giải pháp cuối cùng	32

1 Định nghĩa bài toán

1.1 Bài toán lựa chọn

Hệ thống được xây dựng nhằm thu thập, xử lý, lưu trữ và phân tích dữ liệu thị trường tiền mã hóa từ sàn giao dịch Binance theo thời gian thực. Trong bối cảnh thị trường tiền mã hóa hoạt động liên tục 24/7 với mức độ biến động cao, nhà đầu tư cần một công cụ có khả năng tự động theo dõi thị trường, tính toán các chỉ báo kỹ thuật và phát hiện tín hiệu giao dịch kịp thời.

Dự án tập trung vào việc xây dựng một nền tảng phân tích dữ liệu tiền mã hóa, trong đó dữ liệu giao dịch từ Binance được thu thập liên tục, đưa qua hệ thống xử lý dữ liệu lớn, sau đó được làm sạch, tổng hợp và tính toán các chỉ báo phân tích kỹ thuật. Kết quả xử lý được sử dụng để hiển thị trên Dashboard, đồng thời phục vụ hệ thống cảnh báo giao dịch tự động.

Hệ thống giải quyết các luồng nghiệp vụ chính sau:

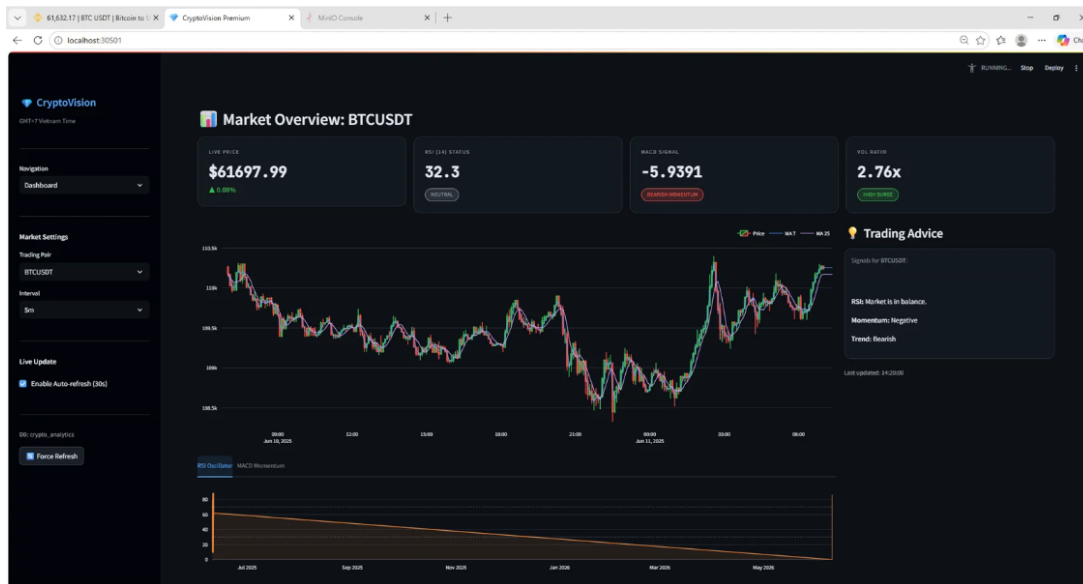
- **Luồng dữ liệu thời gian thực:** Thu thập dữ liệu nến OHLCV, giao dịch đơn lẻ và sổ lệnh từ Binance thông qua WebSocket API; sau đó xử lý bằng Kafka và Spark Streaming để cập nhật dữ liệu gần thời gian thực.
- **Luồng dữ liệu lịch sử:** Thu thập dữ liệu OHLCV lịch sử từ Binance REST API nhằm phục vụ backfill, bổ sung dữ liệu khi hệ thống mới triển khai hoặc sau thời gian gián đoạn.
- **Luồng phân tích kỹ thuật:** Làm sạch dữ liệu, chuẩn hóa timestamp, loại bỏ dữ liệu lỗi/trùng lặp và tính toán các chỉ báo như MA, RSI, MACD, Bollinger Bands, ATR, Volume Ratio và VWAP.
- **Luồng cảnh báo giao dịch:** Đánh giá dữ liệu đã xử lý theo các quy tắc cảnh báo do người dùng cấu hình, từ đó phát hiện tín hiệu BUY, SELL hoặc WATCH và gửi thông báo qua Email hoặc Webhook.
- **Luồng trực quan hóa và giám sát:** Hiển thị biểu đồ giá, chỉ báo kỹ thuật, lịch sử cảnh báo trên Dashboard; đồng thời giám sát trạng thái hệ thống thông qua Kafka, Spark, và Streamlit.

1.2 Phân tích mức độ phù hợp với dữ liệu lớn

Hệ thống phù hợp với bài toán dữ liệu lớn vì đáp ứng đầy đủ các đặc trưng cơ bản của Big Data:

- **Volume:** Dữ liệu thị trường tiền mã hóa có khối lượng lớn, bao gồm dữ liệu nến OHLCV, dữ liệu giao dịch đơn lẻ và dữ liệu sổ lệnh. Với nhiều cặp giao dịch được theo dõi đồng thời, lượng dữ liệu tích lũy tăng nhanh theo thời gian và cần được lưu trữ, xử lý bằng các công nghệ phân tán.
- **Velocity:** Thị trường tiền mã hóa thay đổi liên tục theo thời gian thực. Dữ liệu từ Binance được sinh ra với tốc độ cao, đặc biệt ở các cặp có khối lượng giao dịch lớn như BTCUSDT và ETHUSDT. Vì vậy, hệ thống cần sử dụng Kafka làm message broker và Spark Structured Streaming để tiếp nhận, xử lý luồng dữ liệu với độ trễ thấp.
- **Variety:** Dữ liệu đầu vào có nhiều dạng khác nhau, bao gồm dữ liệu nến OHLCV, tick data, order book và dữ liệu lịch sử từ REST API. Các loại dữ liệu này có cấu trúc, tần suất cập nhật và mục đích sử dụng khác nhau, đòi hỏi hệ thống phải có khả năng thiết kế schema linh hoạt và xử lý theo từng tầng dữ liệu.
- **Veracity:** Dữ liệu thị trường có thể chứa bản ghi trùng lặp, giá trị null, timestamp sai lệch hoặc dữ liệu chưa đóng nến. Do đó, hệ thống cần có các bước kiểm tra chất lượng dữ liệu như deduplication, type casting, null filtering, sanity validation và chuẩn hóa thời gian trước khi đưa vào tầng phân tích.
- **Value:** Dữ liệu sau khi xử lý mang lại giá trị thực tế cho người dùng thông qua biểu đồ phân tích kỹ thuật, tín hiệu giao dịch, cảnh báo tự động và dữ liệu tổng hợp đa khung thời gian. Hệ thống giúp nhà đầu tư giảm việc theo dõi thủ công và hỗ trợ ra quyết định nhanh hơn trong môi trường thị trường biến động liên tục.

1.3 Phạm vi và giới hạn dự án



Hình 1: Ảnh demo phạm vi dự án

Trong phạm vi đề án, hệ thống tập trung xây dựng pipeline dữ liệu lớn phục vụ phân tích thị trường tiền mã hóa từ sàn Binance. Các chức năng chính bao gồm:

- Thu thập dữ liệu thời gian thực từ Binance WebSocket API cho các loại dữ liệu: OHLCV, tick data và order book.
- Thu thập dữ liệu lịch sử từ Binance REST API để phục vụ quá trình backfill.
- Theo dõi 5 cặp giao dịch chính: BTCUSDT, ETHUSDT, BNBUSDT, SOLUSDT và XRPUSDT.
- Lưu trữ và xử lý dữ liệu theo kiến trúc Streaming Lakehouse kết hợp Medallion Architecture với ba tầng Bronze, Silver và Gold.
- Tính toán các chỉ báo kỹ thuật phổ biến gồm MA(7, 25, 99), RSI(14), MACD, Bollinger Bands, ATR(14), Volume Ratio, VWAP và phân loại mẫu nến.
- Tổng hợp dữ liệu nền 1 phút lên các khung thời gian lớn hơn như 5 phút, 15 phút, 1 giờ, 4 giờ và 1 ngày.
- Xây dựng hệ thống quản lý quy tắc cảnh báo cho phép người dùng tạo, xem, sửa, xóa và bật/tắt alert rules thông qua RESTful API.
- Tự động đánh giá điều kiện cảnh báo và gửi thông báo qua Email hoặc Webhook khi dữ liệu thị trường thỏa mãn rule.
- Hiển thị biểu đồ giá, chỉ báo kỹ thuật, lịch sử cảnh báo và trạng thái hệ thống thông

qua Dashboard.

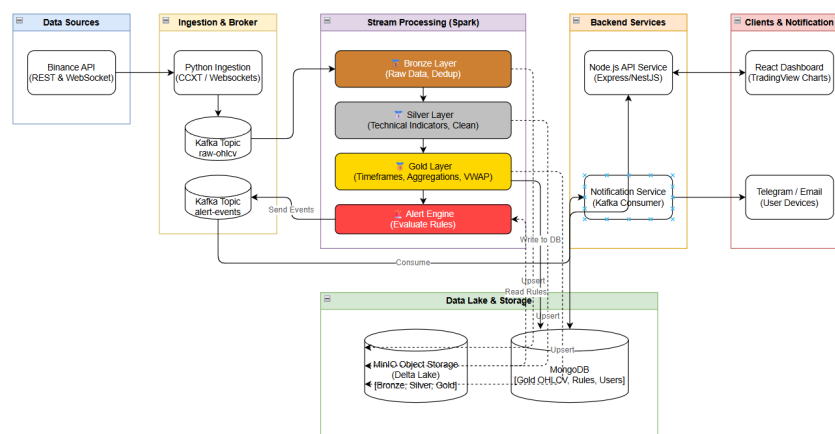
- Giám sát các thành phần chính của hệ thống như Kafka, Spark và Alert Engine bằng Streamlit.

Giới hạn của dự án:

- Hệ thống mới tập trung vào dữ liệu từ sàn Binance, chưa mở rộng sang nhiều sàn giao dịch khác.
- Số lượng cặp tiền được theo dõi giới hạn ở 5 cặp giao dịch phổ biến.
- Các tín hiệu BUY/SELL/WATCH chủ yếu dựa trên rule và chỉ báo kỹ thuật, chưa tích hợp sâu các yếu tố bên ngoài như tin tức, mạng xã hội hoặc dữ liệu on-chain.
- Mô hình Machine Learning dự đoán xu hướng giá mới được định hướng/xây dựng ở mức thử nghiệm, chưa phải thành phần chính để ra quyết định giao dịch.
- Hệ thống được triển khai trong phạm vi đồ án nên còn chịu giới hạn về tài nguyên hạ tầng, đặc biệt khi chạy đồng thời Kafka, Spark, MinIO, MongoDB, APScheduler, và Streamlit trên môi trường cục bộ hoặc cluster nhỏ.

2 Kiến trúc và thiết kế

2.1 Kiến trúc tổng thể



Hình 2: Kiến trúc tổng thể Streaming Lakehouse của hệ thống Cryptocurrency Analytics Platform

Hệ thống áp dụng kiến trúc Streaming Lakehouse kết hợp với mô hình tổ chức dữ liệu Medallion Architecture nhằm đáp ứng yêu cầu xử lý dữ liệu thị trường tiền mã hóa theo

thời gian thực.

Khác với kiến trúc xử lý batch truyền thống, Streaming Lakehouse cho phép dữ liệu liên tục đi từ nguồn phát sinh đến tầng phân tích với độ trễ thấp. Đồng thời, việc sử dụng Lakehouse giúp kết hợp ưu điểm của Data Lake (khả năng lưu trữ dữ liệu lớn, linh hoạt) và Data Warehouse (quản lý schema, đảm bảo tính nhất quán dữ liệu).

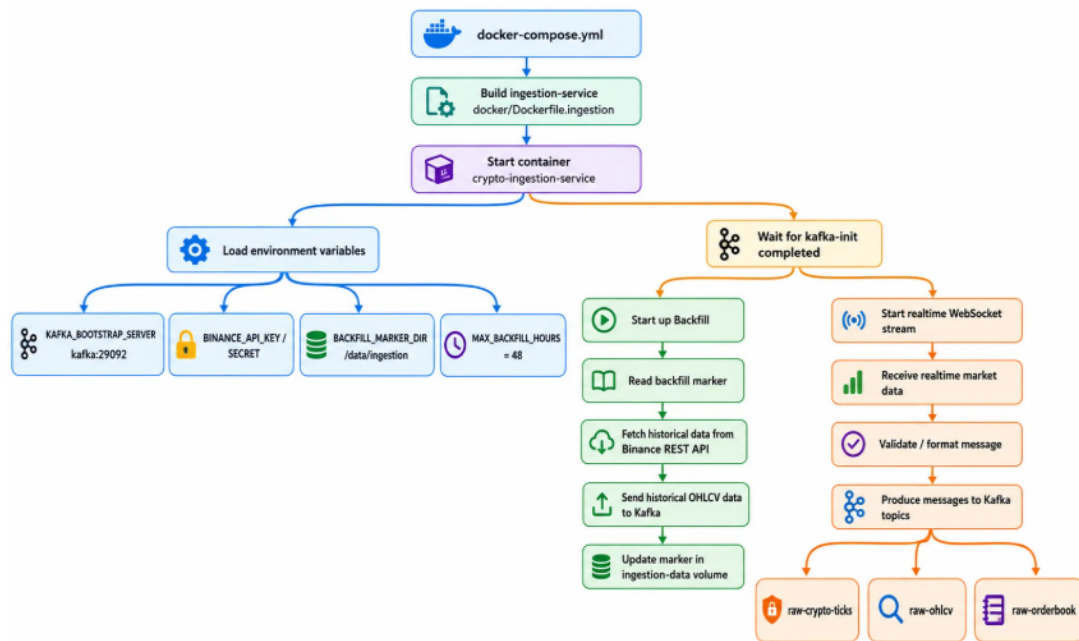
Các tầng chính:

Layer	Công nghệ	Vai trò
Data Source	Binance API	Cung cấp dữ liệu thị trường
Ingestion Layer	Python Producer, Kafka	Thu thập và truyền tải dữ liệu
Processing Layer	Spark Structured Streaming	Xử lý, làm sạch, tính toán
Storage Layer	Delta Lake + MinIO	Lưu trữ dữ liệu Lakehouse
Serving Layer	FastAPI, MongoDB, Dashboard	Truy vấn và cảnh báo
Monitoring Layer	Streamlit	Theo dõi hệ thống

2.2 Data Ingestion Layer

Data Ingestion Layer chịu trách nhiệm thu thập dữ liệu thị trường từ Binance và đưa vào hệ thống xử lý thông qua Apache Kafka.

Hệ thống sử dụng hai nguồn dữ liệu chính:



Hình 3: Chi tiết Data Ingestion Layer

2.2.1 Binance WebSocket API

Dùng cho dữ liệu thời gian thực:

- OHLCV Candle
- Tick Trade
- Order Book

Các luồng WebSocket được duy trì liên tục để nhận dữ liệu ngay khi thị trường thay đổi.

Ví dụ:

BTCUSDT kline

ETHUSDT trade

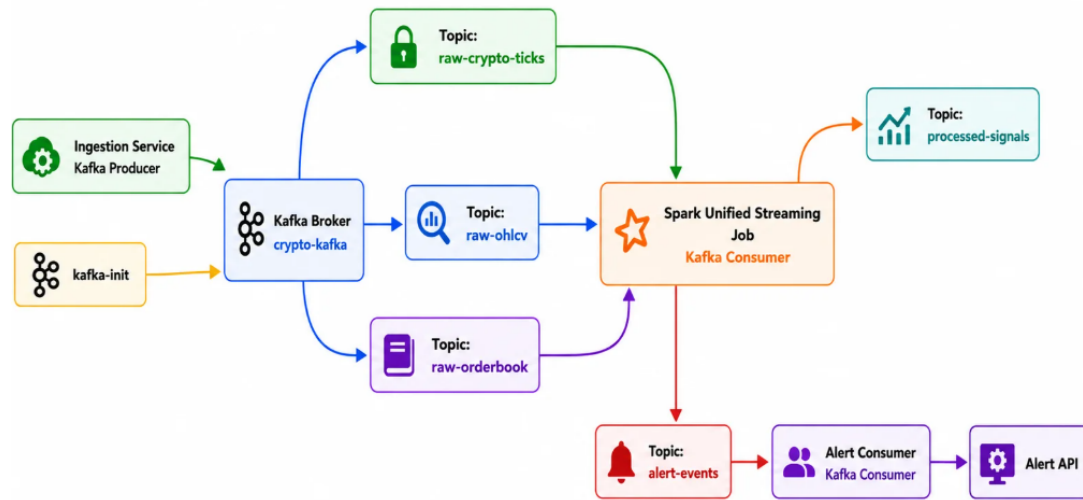
BNBUSDT orderbook

2.2.2 Binance REST API

REST API được sử dụng cho:

- lấy dữ liệu lịch sử
- backfill khi hệ thống mới triển khai
- phục hồi dữ liệu thiếu khi pipeline gặp lỗi

2.2.3 Kafka Message Broker



Hình 4: Chi tiết Messaging Layer

Sau khi thu thập, Producer đẩy dữ liệu vào Kafka.

Kafka đóng vai trò:

- Buffer giữa ingestion và processing.
- Giảm phụ thuộc giữa Binance API và Spark.
- Cho phép scale consumer độc lập.
- Đảm bảo không mất dữ liệu khi Spark restart.

Thiết kế topic:

Topic	Dữ liệu	Retention
raw-crypto-ticks	Giao dịch đơn lẻ	7 ngày
raw-ohlcv	Nến OHLCV	7 ngày
raw-orderbook	Sổ lệnh	1 ngày
processed-signals	Tín hiệu xử lý	30 ngày
alert-events	Sự kiện cảnh báo	30 ngày

2.3 Processing & Lakehouse Layer

Processing Layer là thành phần trung tâm của hệ thống, được xây dựng bằng:

- Apache Spark Structured Streaming
- Delta Lake

- MinIO Object Storage

Spark đọc dữ liệu liên tục từ Kafka và xử lý theo mô hình micro-batch.

Pipeline chính:



Hình 5: Chi tiết Processing Layer

2.3.1 Luồng xử lý Medallion Architecture

Bronze Layer – Raw Data Mục tiêu:

Lưu toàn bộ dữ liệu gốc từ Kafka.

Các bước:

1. Parse JSON theo schema.
2. Gắn Kafka metadata: topic, partition, offset, ingest timestamp.
3. Validate dữ liệu cơ bản.
4. Loại bỏ dữ liệu trùng.
5. Ghi Delta Lake.

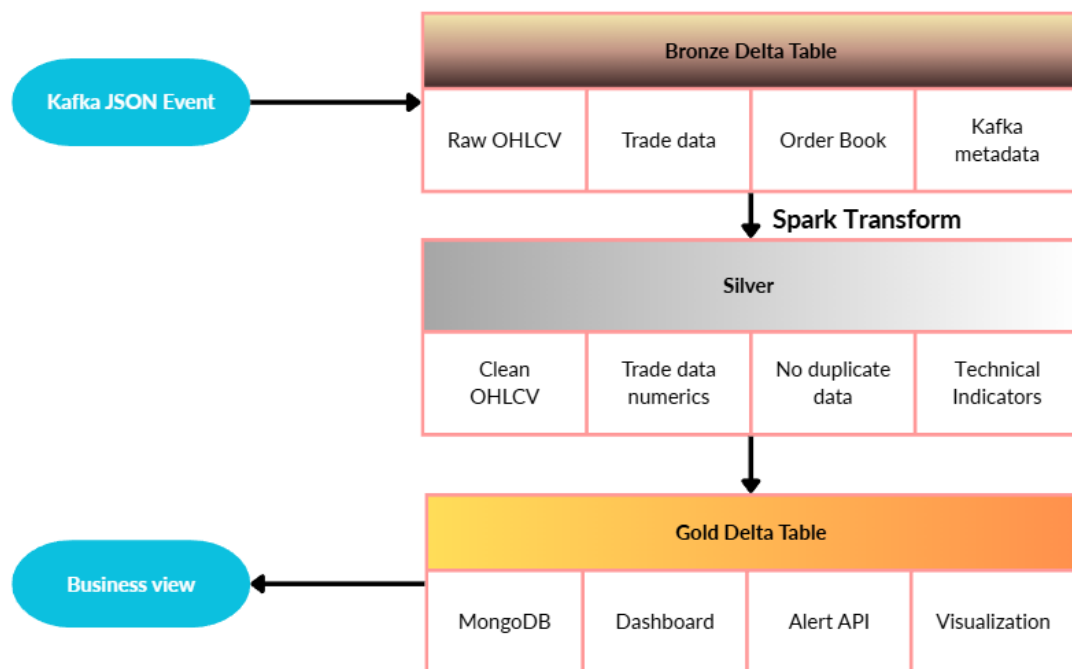
Storage:

s3a://crypto-lake/lakehouse/bronze/ohlcv

Partition:

- symbol
- partition_date

Silver Layer – Cleaned & Enriched Data Silver chịu trách nhiệm biến dữ liệu thô thành dữ liệu phân tích.



Hình 6: Chi tiết Medallion Architecture

Các chỉ báo tính toán:

Indicator	Ý nghĩa
MA7/25/99	Xu hướng giá
RSI	Quá mua/quá bán
MACD	Động lượng
Bollinger Bands	Biến động
ATR	Độ biến động
Volume Ratio	Sức mạnh giao dịch
Candle Pattern	Mẫu nến

Gold Layer – Aggregated Data Gold phục vụ truy vấn nhanh và dashboard.

Nhiệm vụ:

Tổng hợp nhiều timeframe:

1 phút - 5 phút - 15 phút - 1 giờ - 4 giờ - 1 ngày

Tính:

- VWAP
- Indicator cấp cao
- Metadata coin

Output:

- Delta Lake
- MongoDB

2.3.2 Thiết kế Schema dữ liệu

Hệ thống chia schema theo từng tầng.

Bronze Schema Bao gồm:

- symbol
- open_time
- close_time
- open/high/low/close
- volume
- trade_count
- source

→ 17 trường.

Silver Schema Bổ sung:

- Technical Indicators
- Processing metadata

→ 23 trường.

Gold Schema Bổ sung:

- timeframe
- window_start
- window_end
- VWAP

→ 24 trường.

2.3.3 Cơ chế xử lý Streaming

Spark Structured Streaming được sử dụng để đảm bảo:

Exactly-once Processing Thông qua:

- Kafka offset tracking
- Spark checkpoint
- Delta Lake transaction

Fault Tolerance Khi service lỗi:

- Kafka giữ message.
- Spark đọc lại checkpoint.
- Delta tránh ghi trùng.

Optimization Delta Lake hỗ trợ:

- ACID Transaction
- Schema Enforcement
- OPTIMIZE
- Z-Ordering
- VACUUM

2.4 Serving & Alert Layer

Serving Layer phục vụ dữ liệu đã xử lý cho người dùng.

Bao gồm:

2.4.1 FastAPI Alert Engine

Chức năng:

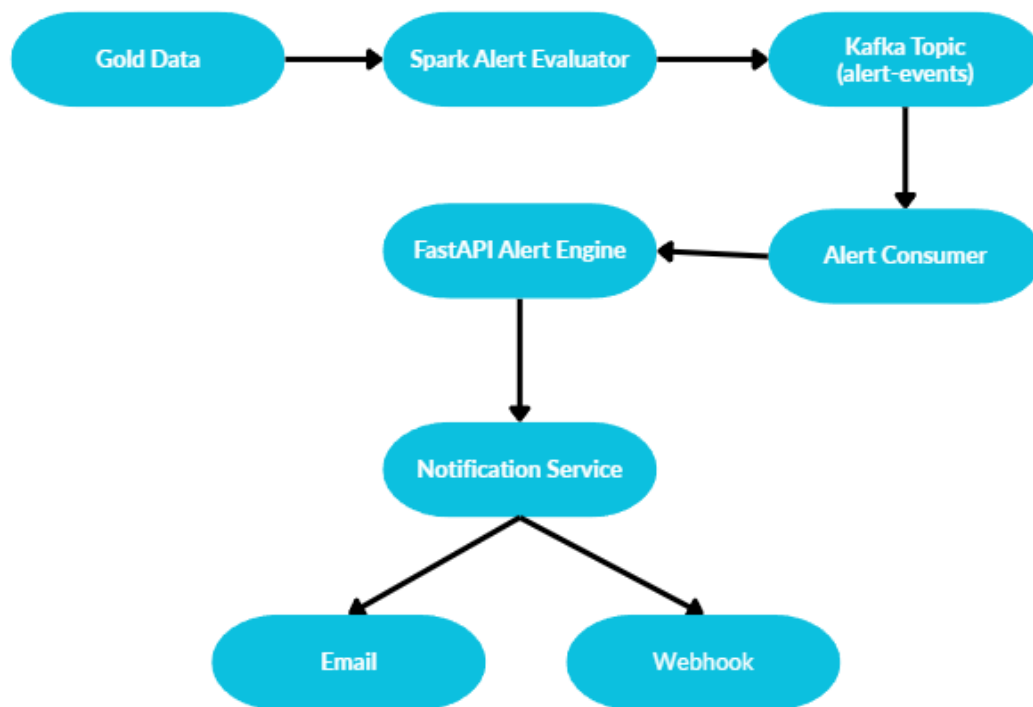
- Tạo Alert Rule
- Cập nhật Rule
- Xóa Rule
- Bật/tắt Rule

Rule lưu trong:

MongoDB

2.4.2 Alert Evaluation Flow

Luồng:



Hình 7: Chi tiết Alert Layer

Output:

- BUY
- SELL
- WATCH

2.4.3 Dashboard Visualization

Dashboard hiển thị:

- Candlestick Chart
- RSI
- MACD
- Volume Ratio
- Alert History
- Trading Signal

2.5 Monitoring Layer

Monitoring giúp kiểm tra trạng thái toàn hệ thống.

Streamlit thu metrics từ:

- Kafka Exporter
- Spark Metrics
- Alert Engine
- Node Exporter

Các metric:

Thành phần	Theo dõi
Kafka	Consumer lag, throughput
Spark	Batch duration, memory
Alert Engine	Request latency
Server	CPU/RAM/Disk

Streamlit Dashboard:

1. Real-time Crypto Prices
2. Kafka & Spark Performance

2.6 APScheduler Orchestration

Apache APScheduler chịu trách nhiệm điều phối pipeline.

Các DAG:

DAG	Vai trò
crypto_bronze_ingestion	Theo dõi ingest
crypto_silver_processing	Clean data
crypto_gold_aggregation	Aggregate
crypto_alert_evaluation	Evaluate rules
crypto_backfill	Load historical data
crypto_quality	Check data quality
crypto_cleanup	Optimize storage

2.7 Deploy Architecture

Hệ thống triển khai container hóa.

Docker:

- Kafka

- Spark
- MongoDB
- MinIO
- APScheduler
- FastAPI

Kubernetes chia namespace:

Namespace	Thành phần
kafka-system	Kafka, MinIO, MongoDB
spark-system	Spark Jobs
apscheduler-system	APScheduler
app-system	Alert Engine
monitoring-system	Streamlit

Helm Chart quản lý:

- infra
- application

3 Chi tiết triển khai

3.1 Phân hệ thu thập dữ liệu (Data Ingestion)

Phân hệ Data Ingestion chịu trách nhiệm kết nối tới Binance, thu thập dữ liệu thị trường và đưa dữ liệu vào hệ thống xử lý thông qua Apache Kafka. Thành phần này được phát triển bằng Python, hoạt động như một Kafka Producer trung gian giữa nguồn dữ liệu bên ngoài và pipeline xử lý phía sau.

Hệ thống triển khai hai cơ chế thu thập dữ liệu nhằm đáp ứng cả nhu cầu xử lý thời gian thực và dữ liệu lịch sử.

Đối với dữ liệu thời gian thực, hệ thống sử dụng Binance WebSocket API để duy trì kết nối liên tục tới sàn giao dịch. WebSocket Producer đăng ký các stream dữ liệu của các cặp tiền BTCUSDT, ETHUSDT, BNBUSDT, SOLUSDT và XRPUSDT. Khi Binance phát sinh sự kiện mới, dữ liệu được nhận ngay lập tức mà không cần gửi request liên tục.

Các loại dữ liệu được thu thập gồm:

- dữ liệu nền OHLCV (Open, High, Low, Close, Volume);

- dữ liệu giao dịch đơn lẻ (Trade/Tick Data);
- dữ liệu sổ lệnh Order Book.

Dữ liệu nhận được từ WebSocket có dạng JSON. Producer thực hiện bước chuẩn hóa ban đầu như bổ sung thời gian ingest, nguồn dữ liệu và kiểm tra các trường quan trọng trước khi gửi vào Kafka.

Luồng xử lý:

Binance WebSocket - Python Producer - Kafka Topic

Ngoài luồng streaming, hệ thống triển khai REST Producer sử dụng Binance REST API v3 để lấy dữ liệu lịch sử. Thành phần này phục vụ quá trình backfill khi hệ thống mới khởi tạo hoặc cần bổ sung dữ liệu bị thiếu sau khoảng thời gian gián đoạn.

Quá trình backfill được quản lý tự động bởi scheduler, đảm bảo dữ liệu lịch sử và dữ liệu realtime có thể kết hợp trong cùng pipeline.

3.2 Triển khai Kafka Message Queue

Apache Kafka được triển khai làm tầng trung gian truyền tải dữ liệu giữa Data Ingestion Layer và Processing Layer.

Thay vì để Spark đọc trực tiếp từ Binance, Kafka đóng vai trò message buffer giúp hệ thống giảm sự phụ thuộc giữa các thành phần. Khi Spark gặp lỗi hoặc cần restart, dữ liệu vẫn được Kafka lưu giữ và có thể tiếp tục xử lý từ offset trước đó.

Hệ thống triển khai 5 Kafka topics:

Topic	Chức năng
raw-crypto-ticks	Lưu từng giao dịch phát sinh trên Binance
raw-ohlc	Lưu dữ liệu nến OHLCV
raw-orderbook	Lưu trạng thái sổ lệnh
processed-signals	Lưu tín hiệu giao dịch đã xử lý
alert-events	Lưu sự kiện cảnh báo

Mỗi topic được chia thành nhiều partition nhằm tăng khả năng xử lý song song. Chiến lược partition dựa trên symbol giúp dữ liệu của cùng một cặp giao dịch luôn được xử lý đúng thứ tự.

Kafka Producer được cấu hình:

- **acks=all** để đảm bảo message được xác nhận ghi thành công.

- `enable.idempotence=True` để tránh ghi trùng khi retry.
- `compression.type=snappy` để giảm kích thước dữ liệu truyền tải.

Cách cấu hình này giúp pipeline đạt khả năng chịu lỗi tốt hơn trong môi trường streaming.

3.3 Triển khai xử lý dữ liệu với Apache Spark

Apache Spark Structured Streaming là thành phần xử lý dữ liệu chính của hệ thống. Spark Consumer đọc dữ liệu từ Kafka và xử lý theo từng micro-batch.

Một Spark job thống nhất (`unified_streaming.py`) đảm nhiệm toàn bộ pipeline

Spark sử dụng cơ chế `foreachBatch`, trong đó mỗi batch dữ liệu mới được đưa lần lượt qua các tầng Medallion.

3.3.1 Bronze Processing

Bronze Layer là bước đầu tiên trong pipeline, có nhiệm vụ lưu giữ dữ liệu gần giống với nguồn ban đầu.

Quy trình xử lý:

Kafka JSON Message - Parse Schema - Validate Data - Deduplication - Delta Lake Bronze

Khi nhận dữ liệu Kafka, Spark parse JSON theo schema định nghĩa trước. Sau đó hệ thống

```
\begin{itemize}
  \item Kafka topic
  \item Kafka partition
  \item Kafka offset
  \item thời điểm ingest
\end{itemize}
```

Các bản ghi không hợp lệ như thiếu symbol, thiếu timestamp hoặc giá không chuyển đổi

Một vấn đề quan trọng trong streaming là dữ liệu có thể bị gửi lại do retry hoặc Kafka

Khóa định danh:

```
\begin{verbatim}  
symbol + interval + open_time
```

Điều này đảm bảo một cây nến chỉ tồn tại một phiên bản cuối cùng.

3.3.2 Silver Processing

Silver Layer chịu trách nhiệm làm sạch và làm giàu dữ liệu.

Dữ liệu từ Bronze được xử lý qua nhiều bước:

Đầu tiên, Spark thực hiện ép kiểu dữ liệu. Các giá trị giá và volume từ Binance ban đầu ở dạng String được chuyển thành Double để phục vụ tính toán.

Sau đó pipeline kiểm tra tính hợp lệ:

- giá đóng cửa phải lớn hơn 0;
- volume không âm;
- giá cao nhất phải lớn hơn hoặc bằng giá thấp nhất.

Sau khi dữ liệu đạt yêu cầu, Spark sử dụng Window Function để tính toán các chỉ báo kỹ thuật.

Các chỉ báo được triển khai gồm:

Nhóm	Chỉ báo
Trend	MA7, MA25, MA99
Momentum	RSI, MACD
Volatility	Bollinger Bands, ATR
Volume	Volume Ratio
Pattern	Candle Pattern

Các chỉ báo dạng rolling window như MA99 yêu cầu dữ liệu lịch sử. Vì vậy trước khi tính toán, Spark tải thêm 100 bản ghi gần nhất để đảm bảo kết quả chính xác dù batch hiện tại có ít dữ liệu.

Các hàm không có sẵn trong Spark như phân loại mẫu nến được triển khai bằng UDF.

3.3.3 Gold Processing

Gold Layer tạo ra dữ liệu cuối cùng phục vụ dashboard và alert.

Dữ liệu nền 1 phút từ Silver được tổng hợp thành nhiều khung thời gian:

Nguồn	Output
1 phút	5 phút
1 phút	15 phút
1 phút	1 giờ
1 phút	4 giờ
1 phút	1 ngày

Spark sử dụng Window Aggregation để tính:

- open đầu tiên;
- high lớn nhất;
- low nhỏ nhất;
- close cuối cùng;
- tổng volume.

Ngoài các chỉ báo kỹ thuật, Gold Layer bổ sung VWAP (Volume Weighted Average Price), giúp đánh giá mức giá trung bình có xét tới khối lượng giao dịch.

Dữ liệu Gold được ghi ra hai nơi:

Gold Layer

```
|  
+--> Delta Lake  
|  
+--> MongoDB
```

Delta Lake phục vụ phân tích dữ liệu lớn, trong khi MongoDB phục vụ truy vấn nhanh từ Dashboard.

3.4 Triển khai lưu trữ Lakehouse

Hệ thống sử dụng Delta Lake trên MinIO Object Storage để xây dựng Data Lakehouse. MinIO đóng vai trò tương tự Amazon S3 trong môi trường local/Kubernetes.

Delta Lake cung cấp:

- ACID Transaction;
- Schema Enforcement;

- Time Travel;
- OPTIMIZE;
- VACUUM.

Dữ liệu được partition theo:

- symbol
- partition_date
- timeframe

để tăng tốc truy vấn.

3.5 Triển khai Alert Engine và Serving API

Alert Engine được xây dựng bằng FastAPI kết hợp MongoDB.

FastAPI cung cấp REST API cho người dùng quản lý rule:

- Create Rule
- Read Rule
- Update Rule
- Delete Rule
- Enable/Disable Rule

Rule được lưu dưới dạng document trong MongoDB.

Khi dữ liệu Gold mới được sinh ra, Spark Alert Evaluator kiểm tra các rule đang bật.

Nếu điều kiện thỏa mãn:

Gold Data

|

Rule Evaluation

|

alert-events Topic

|

Notification Service

Thông báo được gửi qua:

- Email SMTP/TLS
- Webhook HTTP

3.6 Triển khai Workflow với APScheduler

Apache APScheduler được sử dụng để điều phối các workflow định kỳ.

Các DAG chính:

DAG	Chức năng
crypto_historical_backfill	Nạp dữ liệu lịch sử
crypto_data_quality	Kiểm tra chất lượng
crypto_cleanup	OPTIMIZE và VACUUM
crypto_model_training	Huấn luyện ML

3.7 Triển khai hạ tầng Docker/Kubernetes

Toàn bộ hệ thống được container hóa bằng Docker.

Nhóm xây dựng image riêng cho:

- Spark Application
- Data Ingestion
- Alert Engine

Ở môi trường phát triển, Docker Compose quản lý toàn bộ stack:

- Kafka
- MinIO
- MongoDB
- Spark
- APScheduler
- Streamlit

Khi triển khai mở rộng, Kubernetes được sử dụng để quản lý container.

Các namespace:

Namespace	Thành phần
kafka-system	Kafka, MinIO, MongoDB
spark-system	Spark
app-system	API, Producer
monitoring-system	Streamlit

3.8 Monitoring và kiểm thử hệ thống

Streamlit thu thập metrics từ:

- Kafka
- Spark
- Alert Engine
- Server

Streamlit trực quan hóa:

- Kafka Consumer Lag
- Spark Processing Time
- Memory Usage
- Alert Count

Ngoài ra hệ thống kiểm thử:

- Unit Test cho UDF và Alert Logic.
- Integration Test cho Spark Pipeline.
- Data Quality Check kiểm tra null, duplicate và độ mới dữ liệu.

4 Bài học kinh nghiệm

4.1 Bài học 1: Đảm bảo tính liên tục khi thu thập dữ liệu Binance

4.1.1 Mô tả vấn đề

Trong hệ thống phân tích dữ liệu tiền mã hóa, Data Ingestion Layer đóng vai trò là điểm bắt đầu của toàn bộ pipeline. Thành phần này cần duy trì kết nối liên tục tới Binance WebSocket API để lấy dữ liệu giao dịch, dữ liệu nến OHLCV và Order Book theo thời gian thực.

Tuy nhiên trong quá trình triển khai thực tế, việc phụ thuộc trực tiếp vào API của bên thứ ba gây ra một số vấn đề. Kết nối WebSocket có thể bị ngắt do lỗi mạng, timeout hoặc Binance chủ động đóng connection sau một khoảng thời gian hoạt động dài. Ngoài ra REST API phục vụ backfill dữ liệu lịch sử cũng chịu giới hạn request rate.

Nếu Producer không có cơ chế phục hồi, pipeline phía sau gồm Kafka, Spark Streaming

và Dashboard sẽ không nhận được dữ liệu mới, dẫn tới biểu đồ và tín hiệu cảnh báo không còn phản ánh đúng trạng thái thị trường.

4.1.2 Cách tiếp cận đã thử

Ban đầu hệ thống triển khai Producer theo hướng tạo một kết nối WebSocket cố định và giả định connection luôn tồn tại. Khi kết nối bị đóng, tiến trình Producer vẫn chạy nhưng không còn nhận dữ liệu.

Đối với dữ liệu lịch sử, việc gửi request liên tục tới REST API cũng có thể dẫn đến vượt giới hạn request, khiến Binance trả về lỗi và quá trình backfill bị gián đoạn.

4.1.3 Giải pháp cuối cùng

Nhóm triển khai cơ chế phục hồi tự động cho Data Producer. WebSocket Client được bổ sung logic reconnect khi phát hiện mất kết nối. Producer lưu trạng thái xử lý hiện tại để khi khởi động lại có thể tiếp tục lấy dữ liệu thay vì chạy lại toàn bộ.

Đối với REST API, hệ thống bổ sung cơ chế giới hạn tốc độ request và chia quá trình backfill thành nhiều batch nhỏ.

Luồng xử lý:

Connection Error

|

v

Detect Failure

|

v

Reconnect Binance

|

v

Resume Streaming

Kết quả là tầng Ingestion có thể hoạt động ổn định trong thời gian dài và giảm nguy cơ mất dữ liệu.

4.1.4 Điểm rút ra

Khi xây dựng hệ thống Streaming, không nên giả định nguồn dữ liệu bên ngoài luôn ổn định. Thành phần thu thập dữ liệu cần có khả năng tự phục hồi, retry và lưu trạng thái để đảm bảo pipeline hoạt động liên tục.

4.2 Bài học 2: Đồng bộ trạng thái giữa Kafka và Spark Structured Streaming

4.2.1 Mô tả vấn đề

Spark Structured Streaming đọc dữ liệu liên tục từ Kafka để xử lý qua các tầng Bronze, Silver và Gold. Một yêu cầu quan trọng là hệ thống phải đảm bảo dữ liệu không bị mất hoặc xử lý trùng khi Spark Job gặp lỗi hoặc container restart.

Trong quá trình thử nghiệm, khi Spark Streaming bị dừng đột ngột, nếu không quản lý trạng thái đúng cách, Spark có thể đọc lại dữ liệu cũ từ Kafka hoặc bỏ qua một số message chưa xử lý.

Điều này gây ảnh hưởng lớn vì dữ liệu tài chính yêu cầu tính chính xác cao. Một cây nến bị nhân đôi hoặc thiếu dữ liệu có thể làm sai lệch các chỉ báo MA, RSI, MACD và dẫn đến cảnh báo giao dịch sai.

4.2.2 Cách tiếp cận đã thử

Cách tiếp cận ban đầu là phụ thuộc hoàn toàn vào Kafka Consumer Offset. Spark được cấu hình đọc dữ liệu mới nhất từ Kafka topic.

Cách này giúp pipeline chạy đơn giản hơn nhưng khi Spark restart sẽ gặp vấn đề: Kafka biết message nào đã gửi cho consumer nhưng không biết Spark đã xử lý và ghi thành công xuống Delta Lake hay chưa.

4.2.3 Giải pháp cuối cùng

Nhóm sử dụng Spark Checkpoint kết hợp với Delta Lake Transaction.

Spark lưu lại:

- Kafka offset đã xử lý;
- trạng thái streaming query;

- metadata của từng batch.

Sau mỗi micro-batch, Delta Lake đảm bảo dữ liệu được commit theo transaction.

Luồng:

Kafka Offset

|

v

Spark Checkpoint

|

v

Delta Transaction Log

Khi Spark restart, job đọc checkpoint gần nhất và tiếp tục từ đúng vị trí trước khi lỗi xảy ra.

4.2.4 Kết quả

Pipeline đạt khả năng xử lý gần Exactly Once. Việc restart Spark hoặc container không còn gây mất dữ liệu hoặc ghi trùng bản ghi.

4.2.5 Điểm rút ra

Trong hệ thống streaming, Kafka chỉ đảm nhiệm lưu message. Muốn đảm bảo tính nhất quán cần kết hợp offset management, checkpoint và storage transaction.

4.3 Bài học 3: Thiết kế Lakehouse thay vì lưu toàn bộ dữ liệu vào Database

4.3.1 Mô tả vấn đề

Ban đầu, một phương án đơn giản là ghi toàn bộ dữ liệu sau xử lý vào MongoDB để Dashboard có thể truy vấn trực tiếp.

Tuy nhiên dữ liệu cryptocurrency tăng liên tục theo thời gian. Với nhiều cặp tiền và nhiều loại dữ liệu như OHLCV, Tick, Order Book, kích thước dữ liệu lịch sử tăng rất nhanh.

Nếu MongoDB vừa lưu dữ liệu lịch sử vừa phục vụ Dashboard realtime, hệ thống dễ gặp vấn đề:

- dung lượng database tăng nhanh;
- query chậm;
- chi phí mở rộng cao.

4.3.2 Cách tiếp cận đã thử

Lưu toàn bộ dữ liệu realtime và historical vào MongoDB.

Cách này thuận tiện ở giai đoạn đầu nhưng không phù hợp với dữ liệu lớn vì phần lớn dữ liệu lịch sử ít được truy vấn trực tiếp.

4.3.3 Giải pháp cuối cùng

Nhóm áp dụng kiến trúc Lakehouse.

Delta Lake trên MinIO được sử dụng làm nơi lưu trữ chính

MongoDB chỉ đóng vai trò Serving Database chứa dữ liệu Gold cần truy vấn nhanh.

Kiến trúc:

Delta Lake + Historical Analytics

MongoDB + Dashboard Query

4.3.4 Kết quả

Hệ thống tách biệt rõ:

- lưu trữ dài hạn;
- xử lý phân tích;
- truy vấn realtime.

Chi phí lưu trữ giảm và database serving nhẹ hơn.

4.3.5 Điểm rút ra

Không phải dữ liệu nào cũng nên đưa vào database truy vấn. Với Big Data, cần phân biệt Data Lake và Serving Storage.

4.4 Bài học 4: Quản lý Small Files và tối ưu Delta Lake

4.4.1 Mô tả vấn đề

Spark Structured Streaming ghi dữ liệu liên tục xuống Delta Lake theo từng micro-batch.

Khi batch interval nhỏ, hệ thống sinh ra rất nhiều file Parquet kích thước nhỏ.

Hiện tượng này gọi là Small Files Problem.

Ảnh hưởng:

- đọc dữ liệu chậm;
- metadata Delta Log lớn;
- Spark mất nhiều thời gian lập kế hoạch query.

4.4.2 Cách tiếp cận đã thử

Ban đầu mỗi micro-batch ghi trực tiếp xuống Delta mà không tối ưu.

Khi dữ liệu tăng lên, thời gian đọc bảng Silver và Gold bắt đầu tăng dù dung lượng thực tế chưa quá lớn.

4.4.3 Giải pháp cuối cùng

Nhóm bổ sung chiến lược tối ưu Delta.

Sử dụng partition hợp lý:

- symbol
- partition_date

Áp dụng:

- OPTIMIZE để gom file nhỏ;
- VACUUM để xóa dữ liệu cũ;
- Z-Ordering để tăng tốc truy vấn theo symbol/time.

Các workflow tối ưu được lập lịch bằng APScheduler.

4.4.4 Điểm rút ra

Trong Lakehouse, việc ghi thành công dữ liệu chưa đủ. Cần quản lý vòng đời dữ liệu để hệ thống duy trì hiệu năng khi mở rộng.

4.5 Bài học 5: Tối ưu tính toán Technical Indicator trên Spark

4.5.1 Mô tả vấn đề

Các chỉ báo như MA99, RSI, MACD không chỉ phụ thuộc vào bản ghi hiện tại mà cần dữ liệu lịch sử trước đó.

Khi xử lý streaming theo micro-batch, nếu chỉ tính toán trên dữ liệu batch hiện tại thì kết quả indicator sẽ sai ở các điểm đầu batch.

Ví dụ MA99 cần 99 cây nến trước đó nhưng batch hiện tại có thể chỉ chứa vài cây nến mới.

4.5.2 Giải pháp cuối cùng

Trước khi tính indicator, Spark đọc thêm dữ liệu lịch sử gần nhất từ Silver Layer.

Quy trình:

`New Batch + Historical Window --> Calculate Indicator`

Sau khi tính xong, chỉ dữ liệu mới được ghi trở lại.

4.5.3 Kết quả

Indicator giữa các batch được liên tục, không bị reset khi Spark chạy batch mới.

4.5.4 Điểm rút ra

Streaming không có nghĩa chỉ xử lý dữ liệu mới. Với bài toán time-series cần thiết kế cơ chế giữ ngữ cảnh lịch sử.

4.6 Bài học 6: Thiết kế dữ liệu phục vụ Dashboard và Alert Engine

4.6.1 Mô tả vấn đề

Nếu Dashboard và Alert Engine truy vấn trực tiếp dữ liệu Silver, mỗi request phải tính toán lại indicator hoặc aggregation.

Điều này làm tăng latency.

4.6.2 Giải pháp cuối cùng

Nhóm đưa toàn bộ xử lý nặng xuống Gold Layer.

Gold chuẩn bị sẵn:

- timeframe aggregation;
- indicator;
- VWAP;
- trading metadata.

MongoDB lưu dữ liệu đã tối ưu để API chỉ đọc và trả kết quả.

4.6.3 Kết quả

Dashboard phản hồi nhanh hơn và Alert Engine chỉ tập trung kiểm tra rule thay vì xử lý dữ liệu.

4.6.4 Điểm rút ra

Trong hệ thống dữ liệu lớn, cần đưa tính toán nặng về pipeline xử lý thay vì tăng phục vụ người dùng.

4.7 Bài học 7: Quản lý cấu hình và triển khai Container

4.7.1 Mô tả vấn đề

Hệ thống gồm nhiều thành phần:

Kafka, Spark, MinIO, MongoDB, APScheduler, FastAPI, và Streamlit.

Nếu cấu hình địa chỉ, password, port trực tiếp trong source code, việc chuyển môi trường từ local sang server hoặc Kubernetes rất khó khăn.

4.7.2 Giải pháp cuối cùng

Nhóm tách toàn bộ cấu hình ra:

- environment variables;
- Docker Compose config;
- Kubernetes ConfigMap/Secret.

Ứng dụng chỉ đọc thông qua biến môi trường.

4.7.3 Kết quả

Một Docker Image có thể chạy trên nhiều môi trường mà không cần chỉnh sửa source code.

4.7.4 Điểm rút ra

Hệ thống phân tán cần thiết kế theo hướng “Build once, run anywhere”.

4.8 Bài học 8: Monitoring Streaming Pipeline

4.8.1 Mô tả vấn đề

Pipeline streaming có thể gặp lỗi nhưng container vẫn chạy, ví dụ:

- Kafka consumer lag tăng;
- Spark batch xử lý chậm;
- Alert Engine không sinh cảnh báo.

Nếu chỉ kiểm tra trạng thái container sẽ không phát hiện được lỗi.

4.8.2 Giải pháp cuối cùng

Nhóm triển khai Streamlit.

Theo dõi:

Thành phần	Metric
Kafka	Consumer Lag
Spark	Batch Duration
API	Latency
Server	CPU/RAM